Zellic

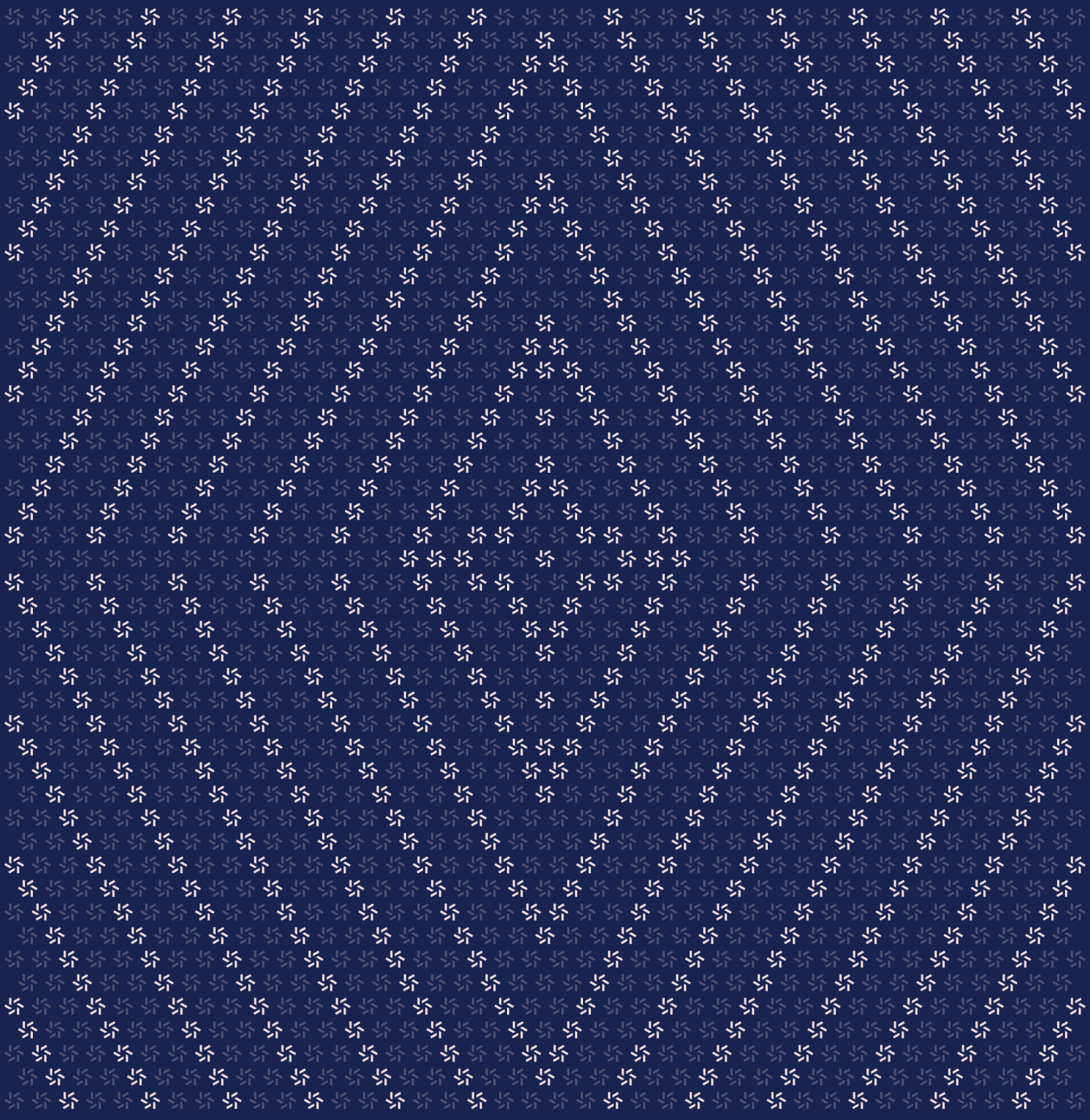
Prepared for
Brent Perreault
Jacob Torres
Paxos

Prepared by
Filipe Alves
Katerina Belotskaia
Zellic

April 20, 2026

PAXG V2

Smart Contract Security Assessment



Contents

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security patch review for Paxos on April 17th, 2026. During this engagement, Zellic reviewed PAXG V2's code for security vulnerabilities, design issues, and general weaknesses in security posture.

We were asked to review a patch to PAXG V2 that upgrades the PAXG to support cross-chain deployment via LayerZero. In section 4. ↗, we provide an overview of the changes.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Does the storage migration preserve the existing layout and execute exactly once on the intended upgrade path?
- Does PAXG correctly enforce frozen-balance semantics given its legacy storage layout?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped PAXG V2 contracts, we discovered one finding, which was informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	0
Informational	1

2. Introduction

2.1. About PAXG V2

Paxos contributed the following description of PAXG V2:

> Paxos builds regulated blockchain and digital asset solutions for global leaders in financial services. In this engagement, PAXG (Paxos Gold) is upgraded from its original Solidity 0.4.24 implementation to the PaxosTokenV2 architecture shared with PYUSD, USDP, and USDG. The new PAXG contract inherits PaxosTokenV2 directly, with two PAXG-specific additions: a one-time storage migration that repositions `paused` and zeroes mislabeled deprecated slots, and a frozen-mapping override that reads and writes from the legacy slot instead of the one declared by `BaseStorage`.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

PAXG V2 Contracts

Type	Solidity
Platform	EVM-compatible
Target	paxos-token-contract-internal
Repository	https://github.com/paxosglobal/paxos-token-contract-internal ↗
Version	`ed52c1155ed628e84c8b7da567481c4717a404e7`
Programs	`contracts/stablecoins/PAXG.sol`
Target	paxos-token-contract-internal
Repository	https://github.com/paxosglobal/paxos-token-contract-internal ↗
Version	`Only changes between 36312f0...2ab1362 (PR #217)`
Programs	`contracts/PaxosTokenV2.sol`

Target	paxos-token-contract-internal
Repository	https://github.com/paxosglobal/paxos-token-contract-internal ↗
Version	`Only changes between 4ab64a7...b818093 (PR #218)`
Programs	`contracts/PaxosTokenV2.sol`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.5 person-days. The assessment was conducted by two consultants over the course of one calendar day.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Filipe Alves
Engineer
filipe@zellic.io ↗

Katerina Belotskaia
Engineer
kate@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

April 17, 2026	Start of primary review period
April 17, 2026	End of primary review period

3. Detailed Findings

3.1. Asymmetric frozen enforcement between the functions `cancelPermits` and `_cancelAuthorization`

Target	EIP2612		
Category	Business Logic	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The function `cancelPermits` in the contract EIP2612 does not validate whether the caller is frozen, making it inconsistent with both the function `_permit` in the same contract and the function `_cancelAuthorization` in EIP3009, which revert with the error `AddressFrozen` when the relevant account is frozen.

```
// contracts/lib/EIP2612.sol#L-114
function cancelPermits(uint256 count) external whenNotPaused {
    if (count == 0 || count > MAX_NONCE_INCREMENT) revert InvalidNonceCount();

    _nonces[msg.sender] += count;
    emit PermitInvalidated(msg.sender, _nonces[msg.sender]);
}
```

In contrast, `_cancelAuthorization` in EIP3009 enforces this check:

```
// contracts/lib/EIP3009.sol#L-319
function _cancelAuthorization(address authorizer, bytes32 nonce,
    bytes memory signature) internal {
   if (_isAddrFrozen(authorizer)) revert AddressFrozen();
    // [...]
}
```

As a result, a frozen account can still invalidate its own outstanding permits, even though it cannot create new ones or cancel EIP-3009 authorizations.

Impact

The inconsistency does not pose a direct security risk, as the actual permit and transfer functions continue to enforce frozen checks, so no token movement can be initiated by a frozen account. However, it creates an uneven enforcement of the frozen-account semantics across related flows.

Recommendations

Add a frozen check to the function `cancelPermits` for consistency with the rest of the frozen-state enforcement.

Remediation

This issue has been acknowledged by Paxos, and a fix was implemented in commit 289229aa ↗.

4. Patch Review

This section documents changes applied to the PaxosTokenV2 contract: PR #217 from commit 36312f00 to commit 2ab13624 and PR #218 from commit 4ab64a74 to commit b818093f. Additionally, it covers the new PAXG contract introduced at commit ed52c115.

4.1. New PAXG contract

This contract inherits PaxosTokenV2, which has been previously audited. The new PAXG contract introduces a storage migration function, `_migratePAXGStorage`, since the previously deployed PAXG has a different storage layout from `BaseStorage` (`PaxosTokenV2`) across slots 4–8.

This is the storage layout of the previous PAXG version:

```
bool private initialized = false; // slot 0 -> identical
mapping(address => uint256) internal balances; // slot 1 -> identical
uint256 internal totalSupply_; // slot 2 -> identical
mapping(address => mapping(address => uint256)) internal allowed; // slot 3 ->
    identical
address public owner; // slot 4 -> repacked with paused and saved to slot 4
address public proposedOwner; // slot 5 -> zeroed out
bool public paused = false; // slot 5 -> migrated to slot 4 -> zeroed out
address public assetProtectionRole; // slot 6 -> zeroed out
mapping(address => bool) internal frozen; // slot 7
address public supplyController; // slot 8 -> zeroed out
address public betaDelegateWhitelister; // slot 9 -> identical
mapping(address => bool) internal betaDelegateWhitelist; // slot 10 ->
    identical
mapping(address => uint256) internal nextSeqs; // slot 11 -> identical
bytes32 public EIP712_DOMAIN_HASH; // slot 12 -> identical
uint256 public feeRate; // slot 13 -> setSupplyControl overwrites it.
address public feeController; // slot 14 -> zeroed out
address public feeRecipient; // slot 15 -> zeroed out
```

This is the `BaseStorage` storage layout:

```
// 0-3 slots are identical
address public ownerDeprecated; // slot 4
bool public paused; // slot 4
address public assetProtectionRoleDeprecated; // slot 5
mapping(address => bool) internal frozen; // slot 6
address public supplyControllerDeprecated; // slot 7
address public proposedOwnerDeprecated; // slot 8
address public betaDelegateWhitelisterDeprecated; // slot 9
mapping(address => bool) internal betaDelegateWhitelistDeprecated; // slot 10
mapping(address => uint256) internal nextSeqsDeprecated; // slot 11
bytes32 public EIP712_DOMAIN_HASH_DEPRECATED; // slot 12
```

```
SupplyControl public supplyControl; // slot 13
uint256[24] __gap_BaseStorage;
```

```
function _migratePAXGStorage() private {
    // solhint-disable-next-line no-inline-assembly
    assembly {
        let slot4Val := sload(4) // owner address (low 20 bytes)
        let slot5Val := sload(5) // proposedOwner (low 20 bytes) + paused (byte
    20)

        let ownerAddr := and(slot4Val,
    0xffffffffffffffffffffffffffffffffffffffff)
        let pausedFlag := and(shr(160, slot5Val), 0xff)

        // Pack owner + paused into slot 4 (BaseStorage layout)
        sstore(4, or(ownerAddr, shl(160, pausedFlag)))

        // Zero deprecated slots that hold mislabeled V1 data.
        // Slot 7 is left alone — it's still the active frozen mapping base
        // (PAXG reads/writes frozen data at keccak256(addr, 7) via overrides).
        sstore(5, 0)  // was proposedOwner+paused →
    assetProtectionRoleDeprecated
        sstore(6, 0)  // was assetProtectionRole  → frozen mapping base (no
    getter)
        sstore(8, 0)  // was supplyController     → proposedOwnerDeprecated
        sstore(14, 0) // was feeController        → in BaseStorage gap
        sstore(15, 0) // was feeRecipient         → in BaseStorage gap
    }
}
```

The migration makes the following storage changes:

- The `owner` address from slot 4 (PAXG) is repacked with `paused` from slot 5 (PAXG) and stored in slot 4.
- Slots 5, 6, 8, 14, and 15 are zeroed out.
- Slot 13 is not updated during migration but will be overwritten by the subsequent `setSupplyControl` call.
- Slot 7, which holds the `frozen` mapping base, is untouched by this function; it is handled separately by the dedicated frozen override functions.

Slots 0–3 and 9–12 remain unchanged.

4.2. PaxosTokenV2 updates

The `_isAddrFrozen`, `_freeze`, and `_unfreeze` functions have been changed from `private` to `internal virtual`.

Additionally, `initialize` has been updated to invoke `_onUpgradeInitialize`, which executes only when upgrading from a V1 contract:

```
function initialize(uint48 initialDelay, address initialOwner, address pauser,
    address assetProtector) public {
    bool wasInitializedV1 = initializedV1;
    uint64 pastVersion = _getInitializedVersion();
    _initialize(pastVersion, initialDelay, initialOwner, pauser,
    assetProtector);
    if (wasInitializedV1 && pastVersion == 0) {
        _onUpgradeInitialize();
    }
}
```

5. Assessment Results

During our assessment on the scoped PAXG V2 contracts, we discovered one finding, which was informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.